

AVL Insertions and Rotations

AVL Trees, Self-Balancing BST, Rotations · Mentor-Led Lab (Student Edition)

Developed by TalenciaGlobal

Difficulty ★★★ Intermediate

Problem

Pattern: self-balancing binary search tree. Insert a sequence of distinct keys into an AVL tree, rebalancing after each insertion, then report the final shape.

Return the level-order traversal of the tree, its height in edges, and the total number of single rotations performed (an LR or RL double rotation counts as two).

What to Compute

- Level Order: the breadth-first traversal of the final AVL tree.
- Height: the height in edges (an empty tree is -1, a single node is 0).
- Rotations: the count of single rotations (LR and RL each contribute two).

Input / Output Format

Input: Line 1: N. Line 2: N distinct integer keys inserted in order.

Output: Three lines: Level Order, Height, Rotations.

Examples

Example 1

Input: 6 10 20 30 40 50 25

Output:

Level Order: 30 20 40 10 25 50

Height: 2

Rotations: 4

Explanation: Inserting 10,20,30 triggers a left rotation; subsequent inserts rebalance again, leaving a height-2 tree rooted at 30 after 4 single rotations.

Example 2

Input: 6 30 20 10 25 40 35

Output:

Level Order: 30 20 40 10 25 35

Height: 2

Rotations: 2

Explanation: Two rotations keep the tree balanced at height 2, rooted at 30 with children 20 and 40.

Constraints

- Keys are distinct integers, inserted in the given order.
- Rebalance whenever a node's balance factor leaves the range $[-1, 1]$.
- Heights are measured in edges.

Sample Run

Sample Input

```
6
10 20 30 40 50 25
```

Sample Output

```
Level Order: 30 20 40 10 25 50
Height: 2
Rotations: 4
```